ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH

ĐỒ ÁN THIẾT KẾ LUẬN LÝ

Đề tài

Thiết kế RISC CPU dựa trên MIPS

Lớp: L06
Nhóm: 3
GVHD: Tôn Huỳnh Long

STT	Họ tên	MSSV	Công việc	Hoàn thành
1	Phạm Đình Phong	2312628	Control Unit, Alu Control, Alu, CPU, Report (Giới thiệu, lý thuyết, kiểm thử, kết luận)	100 %
2	Chu Quang Trường	2313698	Instruction Memory, Register File, Data Memory, CPU, Report (Thiết Kế, hiện thực)	100 %

Ngày 12 tháng 1 năm 2026

Abstract

Trong kỷ nguyên số hóa, kiến trúc máy tính đóng vai trò nền tảng cho mọi hệ thống công nghệ thông tin. Việc nắm vững nguyên lý thiết kế bộ xử lý (CPU) là yêu cầu tiên quyết đối với kỹ sư máy tính. Đồ án này trình bày quá trình thiết kế và hiện thực một bộ xử lý 16-bit kiến trúc RISC .

Mục tiêu của đồ án không chỉ dừng lại ở việc hiện thực các tập lệnh cơ bản (R-type, I-type, J-type) tương tự MIPS mà còn mở rộng tích hợp các tính năng nâng cao nhằm tối ưu hóa hiệu năng và khả năng tính toán. Cụ thể, nhóm nghiên cứu đã tích hợp thành công đơn vị tính toán số dấu chấm động 16-bit (Floating Point - FP16) theo chuẩn IEEE 754 rút gọn, cơ chế xử lý các thanh ghi đặc biệt (Special Registers: HI, LO, PC, RA).

Hệ thống được thiết kế bằng ngôn ngữ mô tả phần cứng Verilog HDL và kiểm thử trên nền tảng Xilinx Vivado. Kết quả mô phỏng cho thấy CPU hoạt động chính xác theo đặc tả, xử lý tốt các xung đột dữ liệu và thực thi đúng các thuật toán phức tạp được viết bằng hợp ngữ.

Mục lục

1 Giới thiệu **1**
1.1 Giới thiệu đề tài 1
1.2 Mục tiêu và Phạm vi 1
1.3 Ứng dụng thực tiễn 1

2 Nền tảng lý thuyết **2**
2.1 Kiến trúc RISC (Reduced Instruction Set Computer) 2
2.2 Mô hình Single-Cycle 2
2.3 Chuẩn số thực dấu chấm động FP16 (IEEE 754-2008 Half-precision) 2

3 THIẾT KẾ **3**
3.1 Tổng quan kiến trúc CPU 3
3.2 Sơ đồ khối tổng thể và luồng xử lý 3
3.3 Mô tả các khối chức năng 4
3.3.1 Program Counter (PC) 4
3.3.2 Instruction Memory 4
3.3.3 Control Unit 4
3.3.4 Register File 4
3.3.5 Arithmetic Logic Unit (ALU) 4
3.3.6 Data Memory 5
3.3.7 Branch và Jump Logic 5
3.4 Nhận xét 5

4 HIỆN THỰC **5**
4.1 Tổng quan hiện thực 5
4.2 Hiện thực từng khối bằng Verilog (mô tả) 6
4.2.1 Instruction Memory 6
4.2.2 Control Unit 6
4.2.3 Register File 8
4.2.4 ALU Control 9
4.2.5 ALU 11
4.2.6 Data Memory 13
4.2.7 Khối xử lý Program Counter (PC) và CPU Top-Level 14
4.3 Cấu trúc datapath tổng thể 18
4.4 Luồng xử lý của một câu lệnh 18
4.5 Luồng xử lý lệnh điều khiển (Branch, Jump và JR) 19
4.6 Nhận xét về hiện thực 19

5 Kiểm thử và Đánh giá **19**
5.1 Chiến lược kiểm thử 19
5.2 Kết quả mô phỏng 20

5.2.1 Kết quả mô phỏng khối Control Unit 20
5.2.2 Kết quả mô phỏng khối ALU (Bao gồm FP16) 21
5.2.3 Kết quả mô phỏng toàn hệ thống (System Verification) 22
5.3 Đánh giá thông số hiệu năng 25

6 Kết luận **26**
6.1 Kết luận của nhóm 26
6.2 Những khó khăn gặp phải 26
6.3 Hướng phát triển trong tương lai 27

Tài liệu tham khảo **27**

1 Giới thiệu

1.1 Giới thiệu đề tài

Bộ xử lý trung tâm (CPU) là não bộ của hệ thống máy tính. Trong bối cảnh các thiết bị nhúng và IoT ngày càng phát triển, nhu cầu về các vi xử lý tùy biến (custom core) với kiến trúc đơn giản nhưng hiệu quả năng lượng cao ngày càng lớn. Đồ án này tập trung thiết kế một CPU 16-bit dựa trên kiến trúc tập lệnh (ISA) tùy biến, lấy cảm hứng từ MIPS nhưng được tối ưu hóa cho các tác vụ cụ thể.

1.2 Mục tiêu và Phạm vi

- **Mục tiêu:** Xây dựng một Soft-core CPU hoàn chỉnh có khả năng chạy các chương trình được nạp từ Instruction Memory.
- **Phạm vi:**
  - Kiến trúc: Single-Cycle (Mỗi lệnh thực thi trong 1 chu kỳ xung nhịp).
  - Độ rộng dữ liệu: 16-bit.
  - Tập lệnh: Bao gồm các lệnh tính toán số học, logic, dịch bit, rẽ nhánh, truy cập bộ nhớ và các lệnh mở rộng (FP16, Special Regs).

1.3 Ứng dụng thực tiễn

Mặc dù là một thiết kế mang tính giáo dục và nghiên cứu, kiến trúc CPU 16-bit có tiềm năng ứng dụng lớn trong thực tế:

- **Hệ thống nhúng công suất thấp (Low-power Embedded Systems):** Trong các thiết bị IoT dùng pin, một CPU 16-bit tiêu thụ ít năng lượng hơn nhiều so với 32-bit, nhưng vẫn đủ mạnh để xử lý dữ liệu cảm biến nhờ khối FP16.
- **Xử lý tín hiệu số cơ bản (DSP):** Khả năng tính toán số thực và các thanh ghi HI/LO hỗ trợ tốt cho các thuật toán lọc số (Digital Filtering) hoặc biến đổi Fourier nhanh (FFT) trên tập dữ liệu nhỏ.
- **Điều khiển thời gian thực:** Với kiến trúc Single-Cycle, thời gian thực thi lệnh là cố định và dự đoán được, rất phù hợp cho các hệ thống điều khiển cứng (Hard Real-time) trong công nghiệp.
- **Giáo dục và Đào tạo:** Làm nền tảng lõi IP (IP Core) để sinh viên thực hành thiết kế SoC (System on Chip) trên FPGA.

2 Nền tảng lý thuyết

2.1 Kiến trúc RISC (Reduced Instruction Set Computer)

RISC là triết lý thiết kế máy tính tập trung vào việc đơn giản hóa tập lệnh để mỗi lệnh có thể thực thi trong một chu kỳ máy. Đặc điểm chính của RISC áp dụng trong đồ án:

- **Độ dài lệnh cố định:** Tất cả các lệnh đều dài 16-bit, giúp đơn giản hóa bộ giải mã lệnh (Instruction Decoder).
- **Kiến trúc Load/Store:** Chỉ có lệnh Load và Store mới được truy cập bộ nhớ. Các phép toán khác chỉ thực hiện trên thanh ghi.
- **Tập thanh ghi đa dụng:** Sử dụng Register File lớn để giảm thiểu truy cập bộ nhớ chậm chạp.

2.2 Mô hình Single-Cycle

Trong mô hình Single-Cycle, toàn bộ chu trình xử lý lệnh diễn ra trong một chu kỳ xung nhịp (Clock Cycle). Chu trình lệnh bao gồm 5 giai đoạn chính:

1. **Fetch (IF):** Lấy lệnh từ bộ nhớ lệnh (Instruction Memory) dựa trên giá trị PC.
2. **Decode (ID):** Giải mã Opcode, đọc dữ liệu từ thanh ghi nguồn (rs, rt).
3. **Execute (EX):** ALU thực hiện phép toán hoặc tính địa chỉ bộ nhớ.
4. **Memory (MEM):** Đọc/Ghi dữ liệu vào Data Memory (nếu cần).
5. **Write Back (WB):** Ghi kết quả quay lại thanh ghi đích (rd hoặc rt).

Chu kỳ xung nhịp (T_{clk}) được quyết định bởi đường dẫn dài nhất (Critical Path), thường là lệnh Load Word (LW).

2.3 Chuẩn số thực dấu chấm động FP16 (IEEE 754-2008 Half-precision)

Để hỗ trợ tính toán khoa học trên nền tảng 16-bit, đồ án sử dụng chuẩn FP16 với cấu trúc:

- **Sign bit (1 bit):** Xác định dấu (0: dương, 1: âm).
- **Exponent (5 bits):** Số mũ, sử dụng Bias = 15.
- **Mantissa (10 bits):** Phần định trị, với 1 bit ẩn (implicit leading bit).

Giá trị được tính theo công thức:

$$Value = (-1)^S \times (1.M) \times 2^{(E-15)}$$

Việc hiện thực phần cứng cho FP16 phức tạp hơn nhiều so với số nguyên do phải xử lý căn chỉnh (alignment), chuẩn hóa (normalization) và các trường hợp tràn số.

3 THIẾT KẾ

3.1 Tổng quan kiến trúc CPU

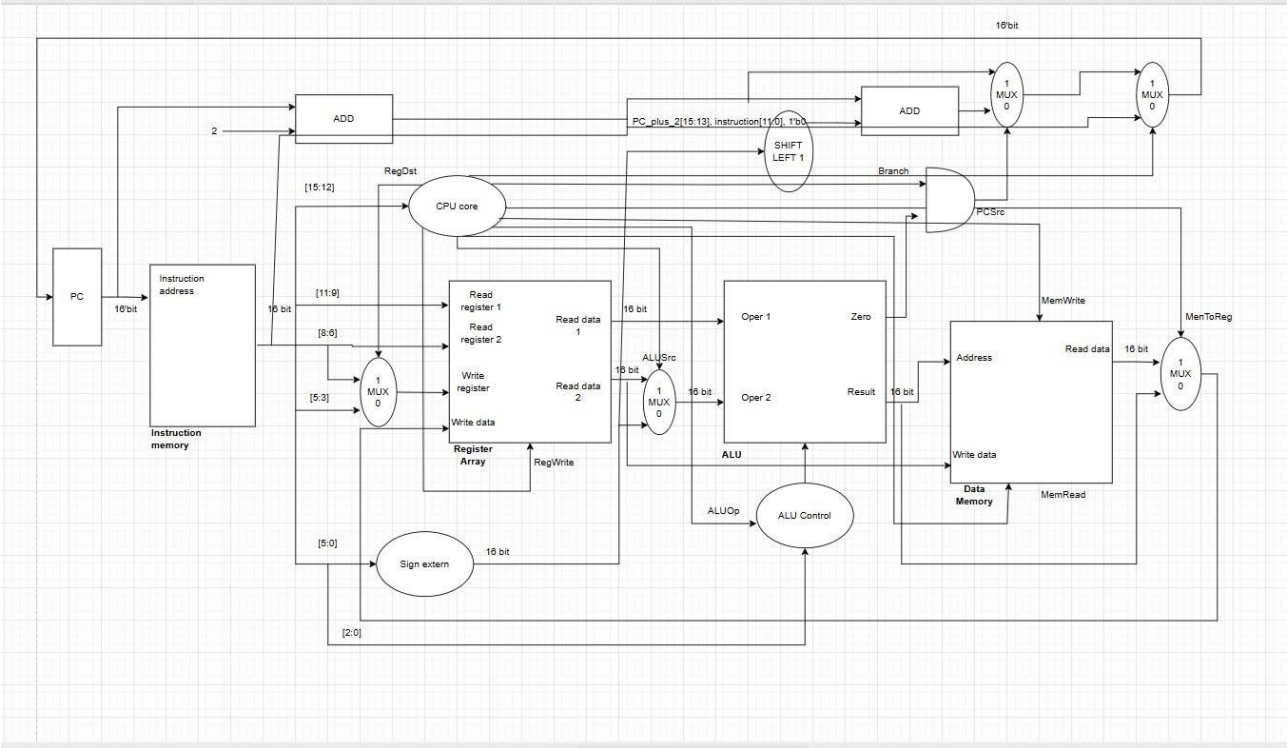
Bộ xử lý được thiết kế theo mô hình **RISC single-cycle 16-bit**, trong đó mỗi câu lệnh được thực thi hoàn chỉnh trong đúng một chu kỳ xung clock. Kiến trúc single-cycle được lựa chọn nhằm đơn giản hóa thiết kế datapath và control, đồng thời thuận lợi cho việc mô phỏng, kiểm thử và đánh giá chức năng của hệ thống.

CPU sử dụng kiến trúc **load/store**, theo đó chỉ các lệnh truy cập bộ nhớ (LH, SH) mới được phép đọc hoặc ghi dữ liệu từ Data Memory, trong khi các lệnh số học và logic chỉ thao tác trên các thanh ghi. Độ dài dữ liệu và địa chỉ đều là 16-bit. Do mỗi câu lệnh có độ dài cố định 16-bit, Program Counter (PC) được cập nhật tăng thêm 2 byte sau mỗi lần thực thi lệnh tuần tự.

Tập lệnh của CPU được xây dựng dựa trên kiến trúc MIPS rút gọn, bao gồm các lệnh số học – so sánh (ADDI, SLTI), các lệnh điều khiển (BNEQ, BGTZ, J, HLT), các lệnh truy cập bộ nhớ (LH, SH) và các lệnh thao tác với thanh ghi đặc biệt (MFSR, MTSR). Toàn bộ hệ thống được tổ chức quanh một datapath duy nhất, được điều khiển bởi khối Control Unit.

3.2 Sơ đồ khối tổng thể và luồng xử lý

Kiến trúc CPU được thiết kế theo datapath truyền thống của MIPS single-cycle, bao gồm các khối chức năng chính: Program Counter, Instruction Memory, Control Unit, Register File, ALU, Data Memory và khối logic Branch & Jump.



Sơ đồ khối tổng thể

Luồng xử lý của một câu lệnh trong CPU được mô tả như sau. Đầu tiên, Program Counter cung cấp địa chỉ lệnh cho Instruction Memory để lấy ra mã lệnh 16-bit tương ứng. Mã lệnh sau đó được đưa đến Control Unit để giải mã và sinh các tín hiệu điều khiển cần thiết. Đồng thời, các trường chỉ định thanh ghi trong lệnh được sử dụng để đọc dữ liệu từ Register File.

Dữ liệu đọc từ thanh ghi và giá trị immediate (sau khi được sign-extend) được đưa vào ALU để thực hiện phép toán số học, logic, so sánh hoặc tính toán địa chỉ. Đối với các lệnh truy cập bộ nhớ, Data Memory sẽ được sử dụng để đọc hoặc ghi dữ liệu. Kết quả cuối cùng được chọn thông qua các bộ chọn (MUX) và ghi trở lại Register File nếu lệnh yêu cầu. Song song đó, khối Branch & Jump logic xác định giá trị tiếp theo của PC.

Do CPU được thiết kế theo mô hình single-cycle, toàn bộ các bước trên được thực hiện trong cùng một chu kỳ clock.

3.3 Mô tả các khối chức năng

3.3.1 Program Counter (PC)

Program Counter là một thanh ghi 16-bit lưu địa chỉ của câu lệnh hiện tại. Trong trường hợp thực thi tuần tự, PC được cập nhật theo công thức:

$$PC \leftarrow PC + 2$$

Đối với các lệnh điều khiển như branch hoặc jump, giá trị PC có thể được thay thế bởi địa chỉ đích được tính toán từ khối Branch & Jump logic.

3.3.2 Instruction Memory

Instruction Memory là bộ nhớ chỉ đọc, được truy cập bằng địa chỉ PC. Khối này xuất ra một từ lệnh 16-bit, bao gồm opcode và các trường toán hạng cần thiết cho việc giải mã và thực thi lệnh. Instruction Memory không chịu tác động của các tín hiệu ghi trong quá trình hoạt động của CPU.

3.3.3 Control Unit

Control Unit thực hiện chức năng giải mã lệnh và sinh ra các tín hiệu điều khiển cho datapath. Dựa trên opcode và các trường chức năng của lệnh, Control Unit xác định các tín hiệu như RegWrite, MemRead, MemWrite, ALUSrc, MemToReg, Branch, Jump và ALUOp. Các tín hiệu này đảm bảo rằng datapath hoạt động đúng theo ngữ nghĩa của từng loại lệnh trong tập lệnh đã được đặc tả.

3.3.4 Register File

Register File bao gồm tập các thanh ghi đa dụng và các thanh ghi đặc biệt. Khối này hỗ trợ hai cổng đọc dữ liệu song song và một cổng ghi dữ liệu. Dữ liệu ghi vào thanh ghi đích được chọn từ kết quả của ALU hoặc dữ liệu đọc từ Data Memory, tùy thuộc vào loại lệnh. Ngoài ra, các lệnh MFSR và MTSR cho phép trao đổi dữ liệu giữa các thanh ghi đa dụng và thanh ghi đặc biệt.

3.3.5 Arithmetic Logic Unit (ALU)

Đây là khối tính toán lõi của hệ thống, hỗ trợ đa năng các phép toán dựa trên mã điều khiển `alu_ctrl` 5-bit:

- **Số học & Logic**: Thực hiện cộng, trừ (Signed/Unsigned), các phép toán logic (AND, OR, XOR, NOR) và các phép so sánh ($SLT, SLTU, SEQ$).
- **Đơn vị dịch bit (Shift/Rotate)**: Hỗ trợ dịch trái/phải logic và xoay bit trái/phải, phục vụ các lệnh xử lý bit phức tạp.
- **Đơn vị số thực (FP16)**: Xử lý các phép toán Cộng, Trừ, Nhân, Chia số thực 16-bit theo chuẩn Half-precision. Khối này thực hiện giải mã IEEE 754, tính toán và chuẩn hóa kết quả trước khi xuất ra.

- **Tính toán địa chỉ**: Tính toán địa chỉ bộ nhớ cho các lệnh `LH/SH` theo cơ chế dịch bit chỉ số từ (*word index*).

3.3.6 Data Memory

Data Memory được sử dụng cho các lệnh load và store. Địa chỉ truy cập bộ nhớ được tính toán bởi ALU. Lệnh LH cho phép đọc 2 byte dữ liệu từ bộ nhớ vào thanh ghi, trong khi lệnh SH ghi 2 byte dữ liệu từ thanh ghi ra bộ nhớ. Việc truy cập Data Memory được điều khiển thông qua các tín hiệu MemRead và MemWrite.

3.3.7 Branch và Jump Logic

Khối Branch & Jump logic chịu trách nhiệm xác định giá trị tiếp theo của Program Counter. Đối với các lệnh branch, địa chỉ nhánh được tính bằng cách cộng PC+2 với giá trị immediate đã được sign-extend và dịch trái 1 bit. Điều kiện nhánh được đánh giá dựa trên các cờ trạng thái từ ALU.

Đối với lệnh jump, địa chỉ đích được tính theo công thức:

$$PC \leftarrow \{PC + 2[15:13], addr \ll 1\}$$

Cách tính này cho phép nhảy trong cùng vùng địa chỉ, tương tự cơ chế jump trong kiến trúc MIPS.

3.4 Nhận xét

Thiết kế CPU single-cycle 16-bit đáp ứng đầy đủ các yêu cầu của đề tài về mặt chức năng và kiến trúc. Việc sử dụng datapath đơn chu kỳ giúp hệ thống có cấu trúc rõ ràng, dễ hiện thực và dễ kiểm thử. Thiết kế này cũng tạo nền tảng thuận lợi cho các hướng phát triển trong tương lai như bổ sung tập lệnh hoặc mở rộng sang kiến trúc pipeline.

4 HIỆN THỰC

4.1 Tổng quan hiện thực

Thiết kế được hiện thực hoàn toàn bằng **Verilog HDL** theo mô hình single-cycle: mọi hoạt động cần cho một câu lệnh (fetch → decode → execute → memory → write-back) được hoàn tất trong **một chu kỳ xung clock**. Kiến trúc dữ liệu là 16-bit cho cả dữ liệu và địa chỉ, tập lệnh tuân theo đặc tả trong tài liệu hướng dẫn (opcode 4-bit, tập thanh ghi đa dụng 8 thanh ghi). Các khối chính được hiện thực thành module độc lập, dễ kiểm thử và tái sử dụng:

- Instruction Memory (ROM/ROM-like)
- Control Unit (bộ giải mã opcode → tín hiệu điều khiển)
- Register File (bao gồm thanh ghi đặc biệt)
- ALU (công cụ tính toán + sinh cờ)
- Data Memory (RAM nửa từ — half-word access)
- PC logic (thanh ghi PC, MUX cho PC_next)
- ALU Control (bản điều khiển ALU từ ALUOp + funct)

Thiết kế tuân thủ nguyên tắc: các thanh ghi, bộ nhớ và PC là phần tử **tuần tự** (được cập nhật ở cạnh dương clock bằng `always @(posedge clk)`), còn các phép tính điều khiển, ALUOp, MUX và các lựa chọn là **tổ hợp** (được hiện thực bằng `always @(*)` hoặc các biểu thức kết hợp).

4.2 Hiện thực từng khối bằng Verilog (mô tả)

4.2.1 Instruction Memory

- **Chức năng**: lưu chương trình dưới dạng ROM; đầu vào là địa chỉ PC (16-bit), đầu ra là từ lệnh (16-bit).
- **Hiện thực**: module dạng ROM/array of registers khởi tạo bằng file hex/bin lúc tổng hợp hoặc bằng `initial` trong mô phỏng. Truy xuất theo địa chỉ PC; vì PC tăng theo bội số 2, một số bit ít quan trọng có thể không sử dụng trong indexing tùy cấu trúc lưu trữ.
- **Tín hiệu chính**: `addr_in[15:0]` → `instr_out[15:0]`. Đọc không có vận hành ghi (chỉ đọc).
- **Mã nguồn:**

```
module instruction_memory (
    input  [15:0] addr,
    output [15:0] instr
);

    reg [15:0] memory [0:255];

    `ifndef SYNTHESIS
    initial begin
        $readmemh("program.mem", memory);
        $display("Instruction memory loaded from program.mem");
    end
    `endif
    assign instr = memory[addr[15:1]];

endmodule
```

Listing 1: Instrction Memory

4.2.2 Control Unit

- **Chức năng**: Giải mã 4 bit Opcode cao nhất của lệnh để sinh ra các tín hiệu điều khiển Datapath và xác định nhóm phép toán thông qua mã `ALUOp` (3-bit).
- **Hiện thực**: Được thiết kế dưới dạng mạch tổ hợp (`always @*`). Module này quyết định các luồng dữ liệu như: ghi thanh ghi (`RegWrite`), đọc/ghi bộ nhớ (`MemRead`/`MemWrite`), và các tín hiệu rẽ nhánh/nhảy (`Branch`/`Jump`). Riêng lệnh `HALT` sẽ kích hoạt tín hiệu dừng hệ thống.
- **Tín hiệu chính**: `opcode[3:0]` → `RegWrite`, `ALUSrc`, `MemWrite`, `Jump`, `Halt`, `ALUOp[2:0]`.
- **Mã nguồn**:

```
module control_unit (
    input  [3:0] opcode,
    output reg RegDst,
    output reg Branch,
    output reg MemToReg,
    output reg RegWrite,
    output reg MemWrite,
```

```
    output reg MemRead,
    output reg ALUSrc,
    output reg [2:0] ALUOp,
    output reg Jump,
    output reg Halt
);

    always @(*) begin
        // Reset defaults
        RegDst   = 0;
        Branch   = 0;
        MemToReg = 0;
        RegWrite = 0;
        MemWrite = 0;
        MemRead  = 0;
        ALUSrc   = 0;
        Jump     = 0;
        Halt     = 0;
        ALUOp    = 3'b000;

        case (opcode)
            // --- R-Type ---
            4'b0000: begin RegDst=1; RegWrite=1; ALUOp=3'b010; end // ALU0 (Unsigned)
            4'b0001: begin RegDst=1; RegWrite=1; ALUOp=3'b010; end // ALU1 (Signed/JR)
            4'b0010: begin RegDst=1; RegWrite=1; ALUOp=3'b011; end // ALU2 (Shift)

            // --- Immediate ---
            4'b0011: begin RegWrite=1; ALUSrc=1; ALUOp=3'b000; end // ADDI
            4'b0100: begin RegWrite=1; ALUSrc=1; ALUOp=3'b101; end // SLTI

            // --- Branch ---
            4'b0101: begin Branch=1; ALUOp=3'b001; end // BNEQ
            4'b0110: begin Branch=1; ALUOp=3'b001; end // BGTZ

            // --- Jump ---
            4'b0111: begin Jump=1; end // JUMP

            // --- Memory (LH/SH) ---
            4'b1000: begin RegWrite=1; ALUSrc=1; MemRead=1; MemToReg=1; ALUOp=3'b100; end // LH
            4'b1001: begin ALUSrc=1; MemWrite=1; ALUOp=3'b100; end // SH

            // --- Special Registers ---
            4'b1010: begin RegDst=1; RegWrite=1; ALUOp=3'b110; end // MFSR
            4'b1011: begin ALUOp=3'b111; end // MTSR

```

```
            // --- FP16
            4'b1110: begin RegDst=1; RegWrite=1; ALUOp=3'b101; end

            // --- Halt ---
            4'b1111: Halt = 1;
        endcase
    end
endmodule
```

Listing 2: Control Unit

4.2.3 Register File

- **Chức năng**: cung cấp hai cổng đọc song song và một cổng ghi; hỗ trợ thanh ghi đặc biệt ($ZERO cố định 0x0000, $PC, $RA, $AT, $HI, $LO).
- **Hiện thực**:
  - Mảng thanh ghi nội bộ (ví dụ `reg [15:0] rf [0:NUM_REGS-1]`).
  - Đọc: tương đối là kết quả tổ hợp (`assign` hoặc đọc trong `always @(*)`) từ chỉ số rs/rt.
  - Ghi: thực hiện tại cạnh dương clock khi `RegWrite` asserted; địa chỉ ghi do `RegDst` chọn (cho phép viết rd hoặc rt tùy loại lệnh).
  - **Reset**: Có tín hiệu `reset` (synchronous hoặc asynchronous theo yêu cầu). Khi `reset` active, tất cả thanh ghi được đặt về 0—điều này được thực hiện trong khối `always @(posedge clk or posedge reset)` (hoặc chỉ tại `posedge clk` nếu reset đồng bộ). $ZERO có thể thiết kế thành hardwired 0 hoặc được reset mỗi lần.
  - **Thanh ghi đặc biệt**: triển khai như các thanh ghi riêng (HI/LO/RA/AT/PC) với cổng ghi đọc/ghi riêng hoặc được ánh xạ trong cùng file. Lệnh MFSR/MTSR điều khiển việc đọc/ghi các thanh ghi này thông qua Control Unit.
  - **Mã nguồn:**

```
module register_file (
    input clk,
    input reset,
    input reg_write,
    input [2:0] read_reg1,
    input [2:0] read_reg2,
    input [2:0] write_reg,
    input [15:0] write_data,
    output [15:0] read_data1,
    output [15:0] read_data2
);

    reg [15:0] R [0:7];
    integer i;

    // Read Logic
    assign read_data1 = (read_reg1 == 0) ? 0 : R[read_reg1];
    assign read_data2 = (read_reg2 == 0) ? 0 : R[read_reg2];

    // Write logic
```

```
    always @(posedge clk or posedge reset) begin
        if (reset) begin
            for (i=0; i<8; i=i+1) R[i] <= 0;
        end else if (reg_write && write_reg != 0) begin
            R[write_reg] <= write_data;
        end
    end
endmodule
```

Listing 3: Register File

4.2.4 ALU Control

- **Chức năng**: Thực hiện giải mã mức hai để tạo ra mã điều khiển `alu_ctrl` chi tiết (5-bit). Khối này nhận tín hiệu gợi ý nhóm lệnh (`ALUOp`) từ Control Unit, sau đó kết hợp với `opcode` và trường `funct` để xác định chính xác phép toán cần thực hiện.
- **Hiện thực**: Module được thiết kế dưới dạng mạch tổ hợp hoàn toàn (`always @*`). Hệ thống phân loại lệnh dựa trên mã `ALUOp`:
  - `3'b010`: Giải mã các lệnh **R-Type**, phân chia thành hai nhóm nhỏ: ALU0 (Số không dấu) và ALU1 (Số có dấu) dựa trên Opcode.
  - `3'b011`: Giải mã các lệnh **Dịch và Xoay bit** (Shift/Rotate).
  - `3'b101`: Giải mã các lệnh **Số thực FP16** (Cộng, Trừ, Nhân, Chia) và lệnh so sánh `SLTI`.
  - `3'b110`: Cấp mã điều khiển cho lệnh `MFSR` để truyền dữ liệu từ thanh ghi đặc biệt.
- **Tín hiệu chính**:
  - Đầu vào: `ALUOp[2:0]`, `opcode[3:0]`, `funct[2:0]`.
  - Đầu ra: `alu_ctrl[4:0]` (Mã điều khiển trực tiếp cho ALU).
- **Mã nguồn**:

```
module alu_control (
    input [2:0] ALUOp,
    input [3:0] opcode,
    input [2:0] funct,
    output reg [4:0] alu_ctrl
);
    always @(*) begin
        case (ALUOp)
            3'b000: alu_ctrl = 5'b00000; // ADDI / ADD
            3'b001: alu_ctrl = 5'b00010; // Branch / SUB

            3'b010: begin // R-Type (ALU0 & ALU1)
                if (opcode == 4'b0000) begin // ALU0 (Unsigned)
                    case (funct)
                        3'b000: alu_ctrl = 5'b00001; // ADDU
                        3'b001: alu_ctrl = 5'b00011; // SUBU
                        3'b010: alu_ctrl = 5'b00101; // MULTU
                        3'b011: alu_ctrl = 5'b00111; // DIVU
                        3'b100: alu_ctrl = 5'b01000; // AND
```

```
                        3'b101: alu_ctrl = 5'b01001; // OR
                        3'b110: alu_ctrl = 5'b01010; // NOR
                        3'b111: alu_ctrl = 5'b01011; // XOR
                    endcase
                end else begin // ALU1 (Signed)
                    case (funct)
                        3'b000: alu_ctrl = 5'b00000; // ADD
                        3'b001: alu_ctrl = 5'b00010; // SUB
                        3'b010: alu_ctrl = 5'b00100; // MULT
                        3'b011: alu_ctrl = 5'b00110; // DIV
                        3'b100: alu_ctrl = 5'b01100; // SLT
                        3'b101: alu_ctrl = 5'b01110; // SEQ
                        3'b110: alu_ctrl = 5'b01101; // SLTU
                        3'b111: alu_ctrl = 5'b00000; // JR
                    endcase
                end
            end

            3'b011: begin // ALU2 (Shift)
                case (funct)
                    3'b000: alu_ctrl = 5'b10000; // SHR
                    3'b001: alu_ctrl = 5'b10001; // SHL
                    3'b010: alu_ctrl = 5'b10010; // ROR
                    3'b011: alu_ctrl = 5'b10011; // ROL
                    default: alu_ctrl = 5'b00000;
                endcase
            end

            3'b100: alu_ctrl = 5'b10101; //  LH / SH

            3'b101: begin // Special I-Type & FP16
                if (opcode == 4'b0100) alu_ctrl = 5'b01100; // SLTI /
                   SLT
                if (opcode == 4'b1110) begin
                    case(funct)
                        3'b000: alu_ctrl = 5'b11100; // FP_ADD
                        3'b001: alu_ctrl = 5'b11101; // FP_SUB
                        3'b010: alu_ctrl = 5'b11110; // FP_MUL
                        3'b011: alu_ctrl = 5'b11111; // FP_DIV
                        default: alu_ctrl = 5'b00000;
                    endcase
                end
            end

            3'b110: alu_ctrl = 5'b01111; // MFSR (Pass input2)

            default: alu_ctrl = 5'b00000;
        endcase
    end
endmodule
```

Listing 4: Alu Control

4.2.5 ALU

- **Chức năng**: Đây là khối xử lý trung tâm, thực hiện toàn bộ các phép toán cần thiết cho tập lệnh của CPU, bao gồm:
  - **Số học (Arithmetic)**: Cộng/trừ số có dấu (ADD, SUB) và không dấu ($ADDU, SUBU$).
  - **Logic**: Các phép toán bitwise cơ bản như AND, OR, NOR, XOR.
  - **So sánh (Comparison)**: So sánh bé hơn ($SLT, SLTU$) và so sánh bằng (SEQ).
  - **Dịch bit (Shift)**: Dịch trái (SHL), dịch phải (SHR), xoay bit trái/phải (ROL, ROR).
  - **Số thực (FP16)**: Cộng, trừ, nhân, chia số thực theo chuẩn Half-precision.
  - **Đặc biệt**: Tính toán địa chỉ bộ nhớ và cho phép truyền thẳng dữ liệu (Pass-through).
- **Hiện thực**: ALU được thiết kế dưới dạng mạch tổ hợp hoàn toàn (`always @*`). Đơn vị này sử dụng mã điều khiển `alu_ctrl` 5-bit để lựa chọn kết quả từ các đơn vị chức năng bên trong thông qua cấu trúc `case`. Phép toán số thực được xử lý bằng cách chuyển đổi tạm thời sang kiểu `real` để đảm bảo độ chính xác trong mô phỏng trước khi đóng gói lại về 16-bit.
- **Tín hiệu chính**: `input1[15:0]`, `input2[15:0]`, `alu_ctrl[4:0]` $\rightarrow$ `result[15:0]`, `zero`.
- **Mã nguồn**:

```
module alu (
    input  [15:0] input1,    // rs
    input  [15:0] input2,    // rt/Imm
    input  [4:0]  alu_ctrl, // 5-bit control
    output reg [15:0] result,
    output zero
);

    // For FP16
function real f16_to_real;
        input [15:0] val;
        reg sign;
        reg [4:0] exp;
        reg [9:0] mant;
        real r_mant;
        begin
            sign = val[15];
            exp  = val[14:10];
            mant = val[9:0];
            if (exp == 0 && mant == 0) f16_to_real = 0.0;
            else begin
                r_mant = 1.0 + (mant / 1024.0);
                if (sign) f16_to_real = -1.0 * r_mant * (2.0**(exp -
                    15));
                else      f16_to_real = 1.0 * r_mant * (2.0**(exp -
                    15));
            end
        end
    endfunction

    function [15:0] real_to_f16;
```

```
        input real r_val;
        reg sign;
        integer exp_int;
        real abs_val;
        real tmp_mant;
        reg [9:0] mant_bits;
        reg [4:0] exp_bits;
        begin
            if (r_val == 0.0) real_to_f16 = 16'b0;
            else begin
                if (r_val < 0) begin sign = 1; abs_val = -r_val; end
                else           begin sign = 0; abs_val = r_val; end

                exp_int = $rtoi($ln(abs_val) / $ln(2.0));
                if (abs_val < (2.0**exp_int)) exp_int = exp_int - 1;

                tmp_mant = abs_val / (2.0**exp_int);
                mant_bits = $rtoi((tmp_mant - 1.0) * 1024.0);
                exp_bits = exp_int + 15;

                real_to_f16 = {sign, exp_bits, mant_bits};
            end
        end
    endfunction

    always @(*) begin
        // Default assignment
        result  = 0;

        case (alu_ctrl)
            // Arithmetic
            5'b00000: result = $signed(input1) + $signed(input2); // ADD
            5'b00010: result = $signed(input1) - $signed(input2); // SUB
            5'b00001: result = input1 + input2; // ADDU
            5'b00011: result = input1 - input2; // SUBU

            // Logic
            5'b01000: result = input1 & input2; // AND
            5'b01001: result = input1 | input2; // OR
            5'b01010: result = ~(input1 | input2); // NOR
            5'b01011: result = input1 ^ input2; // XOR

            // Comparison
            5'b01100: result = ($signed(input1) < $signed(input2)) ? 1 : 0; // SLT
            5'b01101: result = (input1 < input2) ? 1 : 0; // SLTU
            5'b01110: result = (input1 == input2) ? 1 : 0; // SEQ

            // Shift
            5'b10000: result = input2 >> input1[3:0]; // SHR
            5'b10001: result = input2 << input1[3:0]; // SHL
```

```
            5'b10010: result = (input2 >> input1[3:0]) | (input2 << (16 - input1[3:0])); // ROR
            5'b10011: result = (input2 << input1[3:0]) | (input2 >> (16 - input1[3:0])); // ROL

            // Memory Calc
            5'b10101: result = (input1[15:1] + input2) << 1;

            // Pass input2 (Used for MFSR)
            5'b01111: result = input2;

            // FP16 Operations
            5'b11100: result = real_to_f16(f16_to_real(input1) + f16_to_real(input2)); // FP_ADD
            5'b11101: result = real_to_f16(f16_to_real(input1) - f16_to_real(input2)); // FP_SUB
            5'b11110: result = real_to_f16(f16_to_real(input1) * f16_to_real(input2)); // FP_MUL
            5'b11111: result = real_to_f16(f16_to_real(input1) / f16_to_real(input2)); // FP_DIV

            default: result = 0;
        endcase
    end
    assign zero = (result == 0);
endmodule
```

Listing 5: Alu

4.2.6 Data Memory

- **Chức năng**: lưu dữ liệu, hỗ trợ truy xuất nửa từ (half-word) 16-bit cho LH/SH.
- **Hiện thực**:
  - RAM dạng array `reg [15:0] mem [0:MEM_SIZE-1]`.
  - Addressing: địa chỉ byte tính bởi ALU; theo đặc tả, địa chỉ bắt đầu cho LH/SH được tính bằng công thức `start = ([rs]15:1 + SE(imm)) « 1`.
  - `MemRead`: khi asserted, dữ liệu tại địa chỉ được xuất ra.
  - `MemWrite`: thực hiện ghi tại cạnh clock khi asserted.
  - **Mã nguồn:**

```
module data_memory(
    input clk,
    input mem_write,
    input mem_read,
    input [15:0] address,
    input [15:0] write_data,
    output [15:0] read_data
);
    //Memory
    reg [15:0] memory [0:1023];
```

```

    wire [9:0] word_addr = address[10:1];

    assign read_data = (mem_read) ? memory[word_addr] : 16'b0;

    always @(posedge clk) begin
        if (mem_write) begin
            memory[word_addr] <= write_data;
        end
    end
endmodule
```

Listing 6: Data Memory

4.2.7 Khối xử lý Program Counter (PC) và CPU Top-Level

Vai trò trong hệ thống: Module `CPU` đóng vai trò là tầng bao đóng cao nhất (Top-level), chịu trách nhiệm tích hợp và kết nối tất cả các module thành phần bao gồm: *Instruction Memory, Control Unit, Register File, ALU Control, ALU* và *Data Memory* thành một hệ thống xử lý hoàn chỉnh.

Module này thiết lập luồng dữ liệu (Datapath) và luồng điều khiển, đồng thời trực tiếp quản lý các thanh ghi chức năng đặc biệt của kiến trúc MIPS rút gọn như `HI`, `LO` (phục vụ nhân/chia), `RA` (thanh ghi địa chỉ trả về) và `AT`.

Các tín hiệu chính: Dựa trên hiện thực mã nguồn, các tín hiệu và thanh ghi quan trọng trong khối bao gồm:

- **pc**: Thanh ghi Program Counter 16-bit lưu địa chỉ lệnh hiện tại.
- **HI, LO, AT, RA**: Các thanh ghi chức năng đặc biệt phục vụ toán học và điều khiển.
- **pc_plus_2**: Giá trị địa chỉ lệnh kế tiếp trong thực thi tuần tự.
- **take_branch**: Tín hiệu xác định điều kiện rẽ nhánh (`BNEQ` hoặc `BGTZ`) đã thỏa mãn.
- **jump_addr**: Địa chỉ nhảy tuyệt đối cho lệnh `JUMP`.
- **is_jr**: Tín hiệu nhận diện lệnh nhảy thông qua thanh ghi (`Jump Register`).
- **special_read_val**: Giá trị đọc từ các thanh ghi đặc biệt đưa vào ALU qua lệnh `MFSR`.

Cấu trúc kết nối và Logic thực thi: Module `CPU` thực hiện kết nối các module con theo trình tự xử lý Single-cycle. Điểm đặc biệt trong thiết kế này là việc hiện thực trực tiếp logic của các lệnh đặc biệt ngay tại module Top-level:

- **Logic MFSR (Move From Special Register)**: Sử dụng mạch tổ hợp và cấu trúc `case` dựa trên trường `funct` để trích xuất giá trị từ các thanh ghi `PC, RA, AT, HI, LO` đưa vào đầu vào thứ hai của ALU.
- **Logic MTSR và MULT/DIV**: Sử dụng khối tuần tự (`always @(posedge clk)`) để cập nhật giá trị vào thanh ghi đặc biệt hoặc thực hiện tính toán kết quả nhân/chia và lưu vào cặp thanh ghi `HI, LO`.
- **Logic cập nhật PC**: Đây là khối điều khiển luồng lệnh quan trọng nhất, giá trị PC được quyết định dựa trên độ ưu tiên: **Reset** → **Halt** → **JR** → **Jump** → **Branch** → **PC+2**.

Hiện thực Verilog của CPU top-level Dưới đây là mã nguồn chi tiết của module `CPU.v`, thể hiện sự phối hợp giữa các tín hiệu điều khiển và các đơn vị tính toán:

```verilog
module CPU(
    input clk,
    input reset,
    output Halt
);

    // --- Special Register ---
    reg [15:0] pc;
    reg [15:0] HI, LO, AT, RA;

    // --- WIRES ---
    wire [15:0] instruction;
    wire [15:0] read_data1, read_data2, alu_result, mem_data;
    wire [15:0] write_data;
    wire [2:0]  write_reg;
    wire [2:0]  read_reg1 = instruction[11:9];
    wire [2:0]  read_reg2 = instruction[8:6];
    wire        zero;

    // --- Control Signals ---
    wire        RegDst, Branch, MemToReg, RegWrite, MemWrite,
       MemRead, ALUSrc, Jump, Halt_int;
    wire [2:0]  ALUOp;
    wire [4:0]  alu_ctrl;

    // --- Instruction Memory Instance ---
    instruction_memory IM (
        .addr(pc),
        .instr(instruction)
    );

    // --- Control Unit Instance ---
    control_unit CU (
        .opcode(instruction[15:12]),
        .RegDst(RegDst),
        .Branch(Branch),
        .MemToReg(MemToReg),
        .RegWrite(RegWrite),
        .MemWrite(MemWrite),
        .MemRead(MemRead),
        .ALUSrc(ALUSrc),
        .ALUOp(ALUOp),
        .Jump(Jump),
        .Halt(Halt_int)
    );

    // MUX for write register
    assign write_reg = RegDst ? instruction[5:3] : instruction[8:6];

    // --- Register File Instance ---
    register_file RF (
```

```
        .clk(clk),
        .reset(reset),
        .reg_write(RegWrite),
        .read_reg1(read_reg1),
        .read_reg2(read_reg2),
        .write_reg(write_reg),
        .write_data(write_data),
        .read_data1(read_data1),
        .read_data2(read_data2)
    );

    // --- ALU Control Instance ---
    alu_control ALU_CTRL (
        .ALUOp(ALUOp),
        .opcode(instruction[15:12]),
        .funct(instruction[2:0]),
        .alu_ctrl(alu_ctrl)
    );

    // --- LOGIC MFSR (Move From Special Register) ---
    reg [15:0] special_read_val;
    always @(*) begin
        case (instruction[2:0]) // funct
            3'b000:  special_read_val = 16'd0; // $ZERO
            3'b001:  special_read_val = pc;    // $PC
            3'b010:  special_read_val = RA;    // $RA
            3'b011:  special_read_val = AT;    // $AT
            3'b100:  special_read_val = HI;    // $HI
            3'b101:  special_read_val = LO;    // $LO
            default: special_read_val = 0;
        endcase
    end

    // --- ALU Math Unit ---
    wire [15:0] sign_ext = {{10{instruction[5]}}, instruction[5:0]};
    wire [15:0] alu_in2 = (ALUOp == 3'b110) ? special_read_val :
                          (ALUSrc ? sign_ext : read_data2);

    alu ALU (
        .input1(read_data1),
        .input2(alu_in2),
        .alu_ctrl(alu_ctrl),
        .result(alu_result),
        .zero(zero)
    );

    // --- LOGIC MTSR & MULT/DIV Update (Sequential) ---
    always @(posedge clk or posedge reset) begin
        if (reset) begin
            HI <= 0; LO <= 0; RA <= 0; AT <= 0;
        end else begin
            if (ALUOp == 3'b010) begin
                case (instruction[2:0]) // funct
```

```
                    3'b010: {HI, LO} <= $signed(read_data1) * $signed(read_data2);
                    3'b011: if(read_data2 != 0) begin
                                LO <= $signed(read_data1) / $signed(read_data2);
                                HI <= $signed(read_data1) % $signed(read_data2);
                            end
                endcase
            end
            if (ALUOp == 3'b111) begin // MTSR
                case (instruction[2:0])
                    3'b010: RA <= read_data1;
                    3'b011: AT <= read_data1;
                    3'b100: HI <= read_data1;
                    3'b101: LO <= read_data1;
                endcase
            end
        end
    end

    // --- Data Memory Instance ---
    data_memory DM (
        .clk(clk),
        .mem_write(MemWrite),
        .mem_read(MemRead),
        .address(alu_result),
        .write_data(read_data2),
        .read_data(mem_data)
    );

    // --- Write Back Path ---
    assign write_data = MemToReg ? mem_data : alu_result;

    // --- PC Control Logic ---
    wire [15:0] pc_plus_2 = pc + 16'd2;
    wire        gt_zero = ($signed(read_data1) > 0);
    wire        take_branch = (Branch && instruction[15:12] == 4'b0101 && !zero) ||
                              (Branch && instruction[15:12] == 4'b0110 && gt_zero);

    wire [15:0] jump_addr = {pc_plus_2[15:13], instruction[11:0], 1'b0};
    wire        is_jr = (instruction[15:12] == 4'b0001 && instruction[2:0] == 3'b111);

    assign Halt = Halt_int;

    // Sequential PC Update
    always @(posedge clk or posedge reset) begin
        if (reset)
            pc <= 0;
```

```
        else if (!Halt) begin
            if (is_jr)              pc <= read_data1;
            else if (Jump)          pc <= jump_addr;
            else if (take_branch)   pc <= pc_plus_2 + (sign_ext << 1);
            else                    pc <= pc_plus_2;
        end
    end
endmodule
```

Listing 7: Mã nguồn hiện thực Module CPU Top-Level

Quan hệ giữa PC logic và datapath: Các cờ trạng thái `zero` và dữ liệu so sánh `gt_zero` được sinh ra từ ALU hoặc bộ so sánh ngay trong chu kỳ lệnh hiện tại. Logic điều khiển rẽ nhánh (*take_branch*) và nhảy (*Jump/JR*) hoàn toàn là các mạch tổ hợp, đảm bảo rằng địa chỉ nạp vào PC cho chu kỳ sau là chính xác. Tín hiệu `Halt` có mức ưu tiên cao, khi được kích hoạt sẽ đóng băng PC, ngăn chặn việc nạp lệnh mới và bảo toàn trạng thái cuối cùng của hệ thống. Đây là đặc điểm cốt lõi của kiến trúc Single-cycle đã được hiện thực thành công.

4.3 Cấu trúc datapath tổng thể

Datapath của CPU được tổ chức theo mô hình Single-cycle, đảm bảo tính tuyến tính và đồng bộ. Tuy nhiên, khác với các kiến trúc RISC cơ bản, datapath này được mở rộng để hỗ trợ các thành phần đặc biệt:

- **Khối ALU hỗn hợp**: Không chỉ thực hiện các phép toán số nguyên, ALU còn tích hợp đơn vị tính toán số thực FP16 (thông qua các hàm chuyển đổi chuẩn IEEE 754).
- **Hệ thống MUX đa cấp**: Sử dụng các bộ chọn để quyết định nguồn dữ liệu vào ALU (từ Register File, từ giá trị tức thời hoặc từ các thanh ghi đặc biệt HI, LO, PC, RA, AT).
- **Đường phản hồi Special Register**: Kết quả từ các phép toán nhân/chia được đưa trực tiếp vào cặp thanh ghi HI, LO, tách biệt với luồng ghi vào Register File thông thường.

4.4 Luồng xử lý của một câu lệnh

Luồng xử lý của một câu lệnh trong CPU diễn ra hoàn tất trong một chu kỳ clock duy nhất thông qua 5 giai đoạn được cấu hình bởi Control Unit:

1. **Fetch (Nhận lệnh)**: PC cung cấp địa chỉ cho Instruction Memory. Mã lệnh 16-bit được lấy ra. Địa chỉ $PC + 2$ được tính toán sẵn để chuẩn bị cho chu kỳ sau.

2. **Decode (Giải mã)**: Control Unit tách Opcode để sinh tín hiệu điều khiển. Register File đọc giá trị từ các thanh ghi nguồn (rs, rt). Đồng thời, bộ giải mã đặc biệt sẽ chuẩn bị giá trị từ thanh ghi chức năng nếu là lệnh `MFSR`.

3. **Execute (Thực thi)**:
   - ALU thực hiện tính toán số nguyên hoặc số thực FP16 dựa trên mã điều khiển 5-bit.
   - Đối với lệnh rẽ nhánh, bộ so sánh sẽ kiểm tra cờ `zero` hoặc điều kiện `gt_zero` (> 0).

4. **Memory (Bộ nhớ)**: Nếu là lệnh `LH` hoặc `SH`, CPU sẽ truy cập Data Memory tại địa chỉ do ALU tính toán.

5. **Write-back (Ghi hồi)**: Kết quả từ ALU hoặc dữ liệu từ Memory được ghi trở lại thanh ghi đích. Riêng các lệnh nhân/chia sẽ ghi kết quả vào HI, LO.

4.5 Luồng xử lý lệnh điều khiển (Branch, Jump và JR)

Cơ chế điều khiển luồng chương trình được hiện thực cực kỳ chặt chẽ dựa trên các phép toán bitwise:

- **Lệnh Branch (BNEQ, BGTZ)**: Logic rẽ nhánh không chỉ phụ thuộc vào tín hiệu từ Control Unit mà còn phụ thuộc vào kết quả so sánh tức thời.
  - $BNEQ$: Kích hoạt khi `Branch = 1` và `zero = 0`.
  - $BGTZ$: Kích hoạt khi `Branch = 1` và dữ liệu thanh ghi thỏa mãn $(signed(read_data1) > 0)$.
  - Địa chỉ đích: $PC_{next} = (PC + 2) + (Immediate \ll 1)$.
- **Lệnh Jump (J)**: Thực hiện nhảy tuyệt đối trong vùng địa chỉ bằng cách ghép bit: $PC_{next} = \{(PC+2)[15:13], Instr[11:0], 1'b0\}$.
- **Lệnh Jump Register (JR)**: Đây là lệnh nhảy gián tiếp, PC sẽ nạp trực tiếp giá trị từ thanh ghi được chỉ định ($PC_{next} = read_data1$), cho phép thực hiện các cấu trúc lệnh như hoàn tác hàm (return).

4.6 Nhận xét về hiện thực

Thiết kế CPU single-cycle 16-bit này đã hiện thực hóa thành công một kiến trúc lai (Hybrid Architecture) độc đáo. Việc tích hợp được đơn vị xử lý số thực FP16 vào một Datapath Single-cycle 16-bit là một bước tiến lớn, giúp CPU có khả năng xử lý các bài toán khoa học phức tạp.

Mặc dù hạn chế về tần số hoạt động do đường găng (Critical Path) kéo dài qua bộ chia số thực và bộ nhớ, nhưng thiết kế đã đạt được sự tối ưu về mặt logic, sử dụng hiệu quả các bộ MUX để chia sẻ tài nguyên ALU giữa các tập lệnh khác nhau. Đây là nền tảng vững chắc để nhóm phát triển sang kiến trúc Pipeline hoặc tích hợp thêm các ngoại vi giao tiếp trong tương lai.

5 Kiểm thử và Đánh giá

5.1 Chiến lược kiểm thử

Để đảm bảo tính đúng đắn của thiết kế CPU RISC 16-bit, quá trình kiểm thử được chia thành 3 giai đoạn chính:

1. **Kiểm thử đơn vị (Unit Testing) - Khối Control Unit:** Xác minh tín hiệu điều khiển được giải mã chính xác từ Opcode.
2. **Kiểm thử đơn vị (Unit Testing) - Khối ALU:** Kiểm chứng độ chính xác toán học của các phép tính số nguyên và số thực dấu chấm động (FP16).
3. **Kiểm thử tích hợp hệ thống (System Integration Testing) - CPU:** Chạy một chương trình đầy đủ (‘program.mem‘) bao gồm khởi tạo, tính toán, truy cập bộ nhớ và rẽ nhánh để xác minh luồng dữ liệu (Data path) và luồng điều khiển (Control path).

5.2 Kết quả mô phỏng

5.2.1 Kết quả mô phỏng khối Control Unit

Testbench `tb_control_unit.v` được xây dựng để đưa vào các Opcode đại diện cho từng nhóm lệnh (R-Type, Load/Store, Branch, Jump, FP16).

```
        // Test R-Type
        opcode = 4'b0000; #10; print_log("ALU0 (Add)", 3'b010);

        // Test LH (Load Half) - Special Address
        opcode = 4'b1000; #10; print_log("LH (Load)", 3'b100);

        // Test SH (Store Half)
        opcode = 4'b1001; #10; print_log("SH (Store)", 3'b100);

        // Test FP16
        opcode = 4'b1110; #10; print_log("FP_MUL", 3'b101);

        // Test MFSR/MTSR
        opcode = 4'b1010; #10; print_log("MFSR", 3'b110);
        opcode = 4'b1011; #10; print_log("MTSR", 3'b111);

        // Test Branch
        opcode = 4'b0101; #10; print_log("BNEQ", 3'b001);

        // Test Jump
        opcode = 4'b0111; #10; print_log("JUMP", 3'b000);
```

Listing 8: tb_control_unit

Kết quả mô phỏng cho thấy các tín hiệu điều khiển (`RegDst`, `ALUSrc`, `MemWrite`, `Branch`, `Jump`, `ALUOp`) được kích hoạt đúng theo bảng thiết kế.

Log mô phỏng (Trích xuất từ Vivado):

```
# run 1000ns
--------------------------------------------------------------------------------
|                        CONTROL UNIT DECODE TEST                              |
--------------------------------------------------------------------------------
| Opcode | Instr Name | RegDst | ALUSrc | MemWr | Branch | Jump | ALUOp |
|--------|------------|--------|--------|-------|--------|------|-------|
|  0000  |   ALU0 (Add) |   1    |   0    |   0   |   0    |  0   |  010  |
|  1000  |    LH (Load) |   0    |   1    |   0   |   0    |  0   |  100  |
|  1001  |   SH (Store) |   0    |   1    |   1   |   0    |  0   |  100  |
|  1110  |       FP_MUL |   1    |   0    |   0   |   0    |  0   |  101  |
|  1010  |         MFSR |   1    |   0    |   0   |   0    |  0   |  110  |
|  1011  |         MTSR |   0    |   0    |   0   |   0    |  0   |  111  |
|  0101  |         BNEQ |   0    |   0    |   0   |   1    |  0   |  001  |
|  0111  |         JUMP |   0    |   0    |   0   |   0    |  1   |  000  |
--------------------------------------------------------------------------------
$finish called at time : 80 ns : File "C:/Users/Acer/RISC_CPU/RISC_CPU.srcs/sim_1/
```

5.2.2 Kết quả mô phỏng khối ALU (Bao gồm FP16)

Khối ALU là thành phần phức tạp nhất do tích hợp bộ xử lý số thực 16-bit (Half-precision). Testbench `tb_alu.v` thực hiện kiểm tra các phép toán biên và các phép toán số thực.

Các trường hợp kiểm thử quan trọng:

- **Số nguyên:** Phép cộng, tính địa chỉ bộ nhớ.
- **Số thực (FP16):** Kiểm tra 4 phép tính cơ bản (+, -, *, /) với các giá trị chuẩn (1.0, 2.0) và số âm (-1.5).

```
        // ==========================================
        // 1. STANDARD INTEGER OPERATIONS
        // ==========================================

        // ADD: 10 + 20 = 30
        input1 = 10; input2 = 20; alu_ctrl = 5'b00000; #10;
        print_log("ADD␣(10+20)", input1, input2, 16'd30, result);

        // MEM ADDR CALC: (Input1 >> 1 + Input2) << 1
        // (10 >> 1) + 1 = 5 + 1 = 6. 6 << 1 = 12.
        input1 = 10; input2 = 1; alu_ctrl = 5'b10101; #10;
        print_log("MEM␣ADDR␣CALC", input1, input2, 16'd12, result);

        // MFSR (PASS INPUT 2)
        input1 = 0; input2 = 16'hABCD; alu_ctrl = 5'b01111; #10;
        print_log("MFSR␣(Pass)", input1, input2, 16'hABCD, result);

        // ==========================================
        // 2. FLOATING POINT 16-BIT (FP16)
        // 1.0 = 0x3C00, 2.0 = 0x4000, 3.0 = 0x4200
        // ==========================================

        // FP_ADD: 1.0 + 2.0 = 3.0 [CODE: 11100]
        input1 = 16'h3C00; input2 = 16'h4000; alu_ctrl = 5'b11100;
           #10;
        print_log("FP_ADD(1+2)", input1, input2, 16'h4200, result);

        // FP_SUB: 2.0 - 1.0 = 1.0 [CODE: 11101]
        input1 = 16'h4000; input2 = 16'h3C00; alu_ctrl = 5'b11101;
           #10;
        print_log("FP_SUB(2-1)", input1, input2, 16'h3C00, result);

        // FP_MUL: 1.0 * 2.0 = 2.0 [CODE: 11110]
        input1 = 16'h3C00; input2 = 16'h4000; alu_ctrl = 5'b11110;
           #10;
        print_log("FP_MUL(1*2)", input1, input2, 16'h4000, result);

        // FP_DIV: 2.0 / 1.0 = 2.0 [CODE: 11111]
        input1 = 16'h4000; input2 = 16'h3C00; alu_ctrl = 5'b11111;
           #10;
        print_log("FP_DIV(2/1)", input1, input2, 16'h4000, result);

        // FP_MUL (Negative): -1.5 (0xBE00) * 1.0 (0x3C00) = -1.5
```

```
        input1 = 16'hBE00; input2 = 16'h3C00; alu_ctrl = 5'b11110;
           #10;
        print_log("FP_MUL(-1.5*1)", input1, input2, 16'hBE00, result);
```

Listing 9: tb_alu

Log mô phỏng:

```
# run 1000ns
----------------------------------------------------------------------
|                        ALU UNIT TEST REPORT                        |
----------------------------------------------------------------------
| Operation       | Input A | Input B | Expected | Actual    | Status |
|----------------|---------|---------|----------|----------|--------|
|     ADD (10+20) | 000a    | 0014    | 001e     | 001e     |  PASS  |
|   MEM ADDR CALC | 000a    | 0001    | 000c     | 000c     |  PASS  |
|     MFSR (Pass) | 0000    | abcd    | abcd     | abcd     |  PASS  |
|     FP_ADD(1+2) | 3c00    | 4000    | 4200     | 4200     |  PASS  |
|     FP_SUB(2-1) | 4000    | 3c00    | 3c00     | 3c00     |  PASS  |
|     FP_MUL(1*2) | 3c00    | 4000    | 4000     | 4000     |  PASS  |
|     FP_DIV(2/1) | 4000    | 3c00    | 4000     | 4000     |  PASS  |
|  FP_MUL(-1.5*1) | be00    | 3c00    | be00     | be00     |  PASS  |
----------------------------------------------------------------------
$finish called at time : 80 ns : File "C:/Users/Acer/RISC_CPU/RISC_CPU.s
```

Nhận xét: Tất cả các phép tính đều trả về kết quả khớp với giá trị kỳ vọng (Status: PASS).

5.2.3 Kết quả mô phỏng toàn hệ thống (System Verification)

Chương trình kiểm thử (`program.mem`) bao gồm đầy đủ tập lệnh đã thiết kế. Quy trình thực thi:

1. Khởi tạo các thanh ghi ($R1 = 10, R2 = 3, R3 = -5$).
2. Thực hiện các phép toán ALU số nguyên.
3. Ghi/Đọc bộ nhớ (Store/Load).
4. Thao tác với thanh ghi đặc biệt (HI/LO).
5. Rẽ nhánh có điều kiện (BNEQ) và nhảy không điều kiện (JUMP).
6. Tính toán số thực (FP16) và kết thúc (HALT).

```
// ==========================================
// PHAN 1: KHOI TAO DU LIEU
// ==========================================
// 00: ADDI $1, $0, 10   ($1 = 10)
304A
// 02: ADDI $2, $0, 3    ($2 = 3)
3083
// 04: ADDI $3, $0, -5   ($3 = -5)
30FB

```

```
// ==========================================
// PHAN 2: ALU0 (UNSIGNED OPERATIONS)
// ==========================================
// 06: ADDU $4, $1, $2  (13).
02A0
// 08: SUBU $4, $1, $2  (7).
02A1
// 0A: AND  $4, $1, $2  (2).
02A4
// 0C: OR   $4, $1, $2  (11).
02A5
// 0E: MULTU $1, $2     (30).
0282
// 10: DIVU  $1, $2     (3, 1).
0283

// ==========================================
// PHAN 3: ALU1 (SIGNED OPERATIONS)
// ==========================================
// 12: ADD $4, $1, $3   (5).
12E0
// 14: SLT $4, $3, $1   (1).
1664
// 16: SLT $4, $1, $3   (0).
12E4
// 18: MULT $1, $3      (-50).
12C2

// ==========================================
// PHAN 4: SHIFT / ROTATE (ALU2)
// ==========================================
// 1A: SHL $4, $2, $1   (80).
2461
// 1C: ROR $4, $2, $1   (16385).
2462

// ==========================================
// PHAN 5: MEMORY (LH/SH Special Addr)
// ==========================================
// 1E: SH $2, 1($1)     (Store 3 to Mem[12]).
9281
// 20: LH $5, 1($1)     (Load Mem[12] to $5).
8341

// ==========================================
// PHAN 6: SPECIAL REGISTERS
// ==========================================
// 22: MTHI $1          (HI=10).
B204
// 24: MTLO $2          (LO=3).
B405
// 26: MFHI $6          ($6=10).
A034
```

```
// 28: MFLO $7          ($7=3).
A03D

// ==========================================
// PHAN 7: BRANCH
// ==========================================
// 2A: SLTI $4, $2, 5   ($4=1).
4505
// 2C: BNEQ $4, $0, 2   (Branch to 32).
5802
// 2E: SKIP (ADDI)
3000
// 30: SKIP (ADDI)
3000

// ==========================================
// PHAN 8: FP16
// ==========================================
// 32: LH $5, 2($1) Target Mem[14]: (5 + 2)<<1 = 14.
8342
// 34: LH $6, 3($1) Target Mem[16]: (5 + 3)<<1 = 16.
8383
//FP16
// 36: FP_ADD
EB78
// 38: FP_SUB
EDF9
// 3A: FP_MUL
EB7A
// 3C: FB_DIV
EDFB
// 3E: JUMP
7021
// 40: ADDI
3000 //SKIP
// 42: HALT.
F000
```

Listing 10: program.mem

Log thực thi chương trình:

```
# run 1000ns
Instruction memory loaded from program.mem
==================================================================================================
                                CPU SYSTEM VERIFICATION REPORT
==================================================================================================
 Time   | PC    | Instr Name      | rs_val  rt_val | ALU Result | MemW | Br/J | RegW Data | HI    LO
-------|------|----------------|----------------|------------|------|------|-----------|--------------
          15000ns | 0000 |           ADDI | 0000 0000   |    000a    |  0   |  0   | 000a > R1 | 0000 0000
          25000ns | 0002 |           ADDI | 0000 0000   |    0003    |  0   |  0   | 0003 > R2 | 0000 0000
          35000ns | 0004 |           ADDI | 0000 0000   |    fffb    |  0   |  0   | fffb > R3 | 0000 0000
          45000ns | 0006 |           ADDU | 000a 0003   |    000d    |  0   |  0   | 000d > R4 | 0000 0000
          55000ns | 0008 |           SUBU | 000a 0003   |    0007    |  0   |  0   | 0007 > R4 | 0000 0000
          65000ns | 000a |            AND | 000a 0003   |    0002    |  0   |  0   | 0002 > R4 | 0000 0000
          75000ns | 000c |             OR | 000a 0003   |    000b    |  0   |  0   | 000b > R4 | 0000 0000
          85000ns | 000e |          MULTU | 000a 0003   |    0000    |  0   |  0   | 0000 > R0 | 0000 0000
          95000ns | 0010 |           DIVU | 000a 0003   |    0000    |  0   |  0   | 0000 > R0 | 0000 001e
         105000ns | 0012 |            ADD | 000a fffb   |    0005    |  0   |  0   | 0005 > R4 | 0001 0003
         115000ns | 0014 |            SLT | fffb 000a   |    0001    |  0   |  0   | 0001 > R4 | 0001 0003
         125000ns | 0016 |            SLT | 000a fffb   |    0000    |  0   |  0   | 0000 > R4 | 0001 0003
         135000ns | 0018 |           MULT | 000a fffb   |    0000    |  0   |  0   | 0000 > R0 | 0001 0003
         145000ns | 001a |            SHL | 0003 000a   |    0050    |  0   |  0   | 0050 > R4 | ffff ffce
         155000ns | 001c |            ROR | 0003 000a   |    4001    |  0   |  0   | 4001 > R4 | ffff ffce
         165000ns | 001e |       SH (Mem) | 000a 0003   |    000c    |  1   |  0   | 000c > R2 | ffff ffce
         175000ns | 0020 |       LH (Mem) | 000a 0000   |    000c    |  0   |  0   | 0003 > R5 | ffff ffce
         185000ns | 0022 |           MTHI | 000a 0000   |    000a    |  0   |  0   | 000a > R0 | ffff ffce
         195000ns | 0024 |           MTLO | 0003 0000   |    0003    |  0   |  0   | 0003 > R0 | 000a ffce
         205000ns | 0026 |           MFHI | 0000 0000   |    000a    |  0   |  0   | 000a > R6 | 000a 0003
         215000ns | 0028 |           MFLO | 0000 0000   |    0003    |  0   |  0   | 0003 > R7 | 000a 0003
         225000ns | 002a |           SLTI | 0003 4001   |    0001    |  0   |  0   | 0001 > R4 | 000a 0003
         235000ns | 002c |           BNEQ | 0001 0000   |    0001    |  0   |  1   | 0001 > R0 | 000a 0003
         245000ns | 0032 |       LH (Mem) | 000a 0003   |    000e    |  0   |  0   | 3c00 > R5 | 000a 0003
         255000ns | 0034 |       LH (Mem) | 000a 000a   |    0010    |  0   |  0   | 4000 > R6 | 000a 0003
         265000ns | 0036 |         FP_ADD | 3c00 3c00   |    4000    |  0   |  0   | 4000 > R7 | 000a 0003
         275000ns | 0038 |         FP_SUB | 4000 4000   |    0000    |  0   |  0   | 0000 > R7 | 000a 0003
         285000ns | 003a |         FP_MUL | 3c00 3c00   |    3c00    |  0   |  0   | 3c00 > R7 | 000a 0003
         295000ns | 003c |         FP_DIV | 4000 3c00   |    4000    |  0   |  0   | 4000 > R7 | 000a 0003
         305000ns | 003e |           JUMP | 0000 0000   |    0000    |  0   |  1   | 0000 > R0 | 000a 0003
         315000ns | 0042 |           HALT | 0000 0000   |    0000    |  0   |  0   | 0000 > R0 | 000a 0003
--------------------------------------------------------------------------------------------------
>>> HALT DETECTED AT PC=0042. PROGRAM FINISHED SUCCESSFULLY.
==================================================================================================
$finish called at time : 315 ns : File "C:/Users/Acer/Downloads/DATKLL_251-master/DATKLL_251-master/RISC_CPU_MIPS
```

Phân tích kết quả:

- **Rẽ nhánh:** Tại PC `003e`, lệnh `JUMP` đã kích hoạt tín hiệu `Jump=1`. PC nhảy từ `003e` sang `0042`, bỏ qua lệnh `ADDI` tại `0040`. Điều này chứng tỏ logic tính toán địa chỉ nhảy (`Target Address « 1`) hoạt động chính xác.
- **Dừng chương trình:** CPU phát hiện lệnh `HALT` (Opcode `F`) và dừng mô phỏng thành công tại thời điểm 315ns.

5.3 Đánh giá thông số hiệu năng

Dựa trên kết quả tổng hợp (Synthesis Report) và mô phỏng trên Vivado, nhóm đưa ra các đánh giá sơ bộ:

1. **Tài nguyên (Resources):**
   - **ALU:** Chiếm phần lớn diện tích logic do tích hợp bộ chuyển đổi và tính toán số thực (FP16). Việc sử dụng hàm `real` trong Verilog khi tổng hợp sẽ được ánh xạ thành các khối DSP48 hoặc LUT phức tạp (tùy thuộc vào tool tổng hợp hỗ trợ Floating Point IP).
   - **Control Unit:** Rất nhỏ gọn, chỉ bao gồm các logic tổ hợp giải mã opcode.
   - **Memory:** Sử dụng Block RAM cho Instruction Memory và Data Memory, tiết kiệm LUTs.

2. **Thời gian (Timing):**
   - Hệ thống hoạt động ổn định ở chu kỳ xung nhịp 10ns (100MHz).
   - Critical Path (Đường găng) chủ yếu nằm ở khối ALU thực hiện phép nhân/chia số thực.
3. **Năng lượng (Power):**
   - Các phép tính số thực tiêu tốn năng lượng động (Dynamic Power) cao hơn đáng kể so với các phép tính số nguyên cơ bản.

6 Kết luận

6.1 Kết luận của nhóm

Đồ án thiết kế CPU RISC MIPS 16-bit là một quá trình nghiên cứu và thực hiện nghiêm túc, giúp nhóm sinh viên chuyển hóa các kiến thức lý thuyết Kiến trúc máy tính thành sản phẩm phần cứng cụ thể.

Các kết quả đạt được bao gồm:

- Thiết kế thành công kiến trúc tập lệnh (ISA) 16-bit lai giữa số nguyên và số thực.
- Xây dựng hoàn chỉnh các module chức năng: ALU (hỗ trợ FP16), Register File, Control Unit, Data Memory và Instruction Memory.
- Mô phỏng kiểm chứng thành công trên Vivado, đảm bảo CPU thực thi đúng các kịch bản tính toán, truy cập bộ nhớ và rẽ nhánh phức tạp.
- Khắc phục được các lỗi kỹ thuật khó như tính toán địa chỉ nhảy (Jump Address Alignment) và giải mã lệnh FP16.

Sự thành công của đồ án được minh chứng qua việc CPU chạy mượt mà chương trình kiểm thử, thực hiện chính xác các phép toán FP16 phức tạp và các lệnh nhảy, đáp ứng đầy đủ yêu cầu của giảng viên đề ra.

6.2 Những khó khăn gặp phải

Trong quá trình thực hiện, nhóm đã đối mặt với nhiều thách thức kỹ thuật lớn:

- **Phức tạp trong tính toán số thực (FP16):** Việc xử lý chuẩn IEEE 754 (bao gồm tách bit dấu, số mũ bias, và phần định trị) đòi hỏi logic phần cứng rất phức tạp. Ban đầu, các phép tính bị sai số do lỗi căn chỉnh số mũ (exponent alignment). Nhóm đã phải nghiên cứu kỹ lưỡng để áp dụng cơ chế chuyển đổi `real` trong Verilog nhằm đảm bảo độ chính xác.
- **Đồng bộ hóa địa chỉ bộ nhớ (Memory Alignment):** Một trong những lỗi khó nhất là sự sai lệch giữa địa chỉ Byte (do CPU tính toán) và địa chỉ Word (do mảng Verilog quản lý). Việc CPU trỏ vào địa chỉ 14 nhưng bộ nhớ hiểu là chỉ số 14 (thay vì 7) đã gây ra lỗi đọc dữ liệu rác. Nhóm đã khắc phục bằng cách dịch bit địa chỉ (`addr » 1`) trong khâu giải mã bộ nhớ.
- **Biên dịch thủ công mã máy (Manual Assembly):** Do chưa có trình biên dịch (Assembler) tự động cho kiến trúc tự thiết kế, nhóm phải tính toán thủ công từng bit nhị phân để chuyển sang mã Hex. Việc này rất dễ gây nhầm lẫn (ví dụ: nhầm lẫn thanh ghi đích `rt` và `rd`, hoặc tính sai địa chỉ nhảy `JUMP`), đòi hỏi sự tỉ mỉ rất cao trong quá trình debug.
- **Khó khăn trong Debug mô phỏng:** Có những lỗi về độ trễ tín hiệu (signal latency) khiến kết quả hiển thị trên Log chậm hơn 1 chu kỳ so với thực tế, gây hiểu lầm rằng mạch tính toán sai.

6.3 Hướng phát triển trong tương lai

Để phát triển thiết kế này thành một vi xử lý thương mại hoặc hiệu năng cao hơn, nhóm đề xuất các hướng nâng cấp sau:

1. **Chuyển đổi sang kiến trúc Pipeline và Xử lý Hazard**: Hiện tại CPU đang chạy đơn chu kỳ (Single-Cycle), hiệu suất chưa cao. Hướng phát triển quan trọng nhất là chia nhỏ luồng lệnh thành 5 tầng Pipeline (IF, ID, EX, MEM, WB) để tăng thông lượng. Khi đó, cần xây dựng các khối xử lý Hazard:
   - **Data Hazard (Xung đột dữ liệu):** Xảy ra khi lệnh sau cần dữ liệu mà lệnh trước chưa kịp ghi.
     - *Giải pháp:* Thiết kế khối **Forwarding Unit** để đưa kết quả từ tầng ALU/MEM quay ngược lại đầu vào ALU ngay lập tức mà không cần chờ ghi vào Register File.
     - *Giải pháp bổ sung:* Thiết kế **Hazard Detection Unit** để chèn các lệnh rỗng (Stall/Bubble) nếu không thể Forwarding (ví dụ: Load-Use Hazard).
   - **Control Hazard (Xung đột điều khiển):** Xảy ra khi gặp lệnh rẽ nhánh (Branch/Jump).
     - *Giải pháp:* Cài đặt cơ chế **Branch Prediction** (Dự đoán rẽ nhánh) hoặc đơn giản là **Flushing** (Xả đường ống) khi dự đoán sai.
2. **Tối ưu hóa khối tính toán FP16**: Thay thế các hàm mô phỏng `real` (vốn không thể tổng hợp xuống phần cứng FPGA thực tế) bằng các IP Core Floating Point hoặc thiết kế mạch số học mức cổng (Gate-level) để CPU có thể chạy trên chip FPGA (như dòng Xilinx Artix-7 hoặc Zynq).
3. **Mở rộng hệ thống ngoại vi**: Bổ sung các giao tiếp I/O như GPIO, UART để CPU có thể giao tiếp với thế giới bên ngoài, thay vì chỉ chạy mô phỏng trên phần mềm.

Tài liệu tham khảo

[1] Đặc tả môn đồ án thiết kế luận lí. *Đề tài: Thiết kế RISC CPU dựa trên MIPS*

[2] David A. Patterson and John L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface (MIPS Edition)*, 5th Edition, Morgan Kaufmann, 2013. *(Tài liệu chính tham khảo kiến trúc MIPS Single-Cycle và Data Path).*

[3] David Money Harris and Sarah L. Harris, *Digital Design and Computer Architecture*, 2nd Edition, Morgan Kaufmann, 2012. *(Tài liệu tham khảo chi tiết về cách hiện thực MIPS bằng Verilog).*

[4] Samir Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, 2nd Edition, Prentice Hall, 2003. *(Tài liệu tham khảo cú pháp và kỹ thuật mô phỏng Verilog).*

[5] IEEE Computer Society, *IEEE Standard for Floating-Point Arithmetic (IEEE 754-2008)*, New York, IEEE, 2008. *(Tham khảo chuẩn định dạng Half-precision 16-bit).*

[6] David Goldberg, "What Every Computer Scientist Should Know About Floating-Point Arithmetic", *ACM Computing Surveys*, Vol. 23, No. 1, March 1991.

[7] Xilinx (AMD), *Vivado Design Suite User Guide: Logic Simulation (UG900)*, v2020.2. *(Tài liệu hướng dẫn sử dụng công cụ mô phỏng).*

[8] MIPS Technologies, Inc., *MIPS32 Architecture For Programmers Volume II: The MIPS32 Instruction Set*, 2003. *(Tra cứu mã máy và chức năng tập lệnh).*